알고리즘1 (2026-1)

7-1. 분리 집합의 표현

국립금오공과대학교 컴퓨터공학부

컴퓨터공학전공

김 경 수

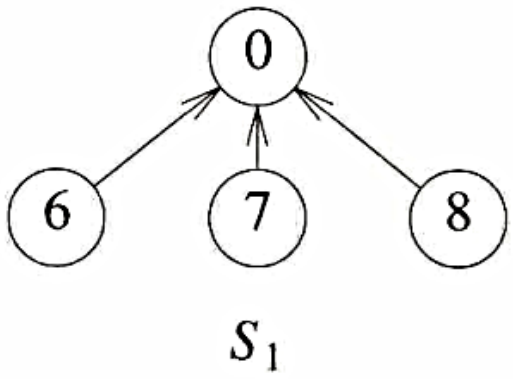
학습목표

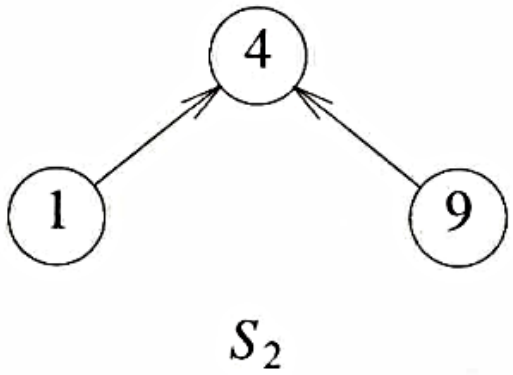
- 분리 집합(Disjoint Set)의 개념을 이해하고, 이를 효과적으로 표현하기 위한 자료구조에 대하여 설명할 수 있다.
- 분리 집합에 대한 Union 연산과 Find 연산의 수행 과정을 이해하고, 이를 구현할 수 있다.
- 분리 집합에 대한 Union 연산과 Find 연산의 수행 시간 복잡도를 분석하고 효율적인 연산을 위한 전략을 이해하여 실제로 구현할 수 있다.
- Kruskal 알고리즘에서 간선 선택 시 사이클이 발생하지 않음을 분리 집합 표현을 이용해서 확인하는 방법을 이해할 수 있다.

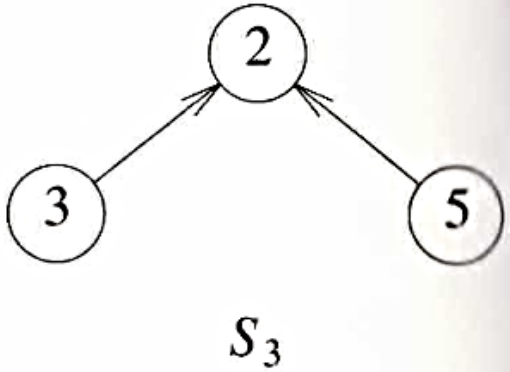
분리 집합의 표현 방법

- 분리 집합(disjoint set)은 서로 어떤 원소도 공통으로 가지지 않는 두 개 이상의 집합을 의미하며, 이들 각각의 집합은 트리로 표현할 수 있다.
- 이 때, 각 집합은 해당 집합을 나타내는 트리의 루트 노드 값으로 구분한다.
- (예) 분리 집합의 표현 방법:

$$S_1=\{0, 6, 7, 8\}, \quad S_2=\{1, 4, 9\}, \quad S_3=\{2, 3, 5\}$$



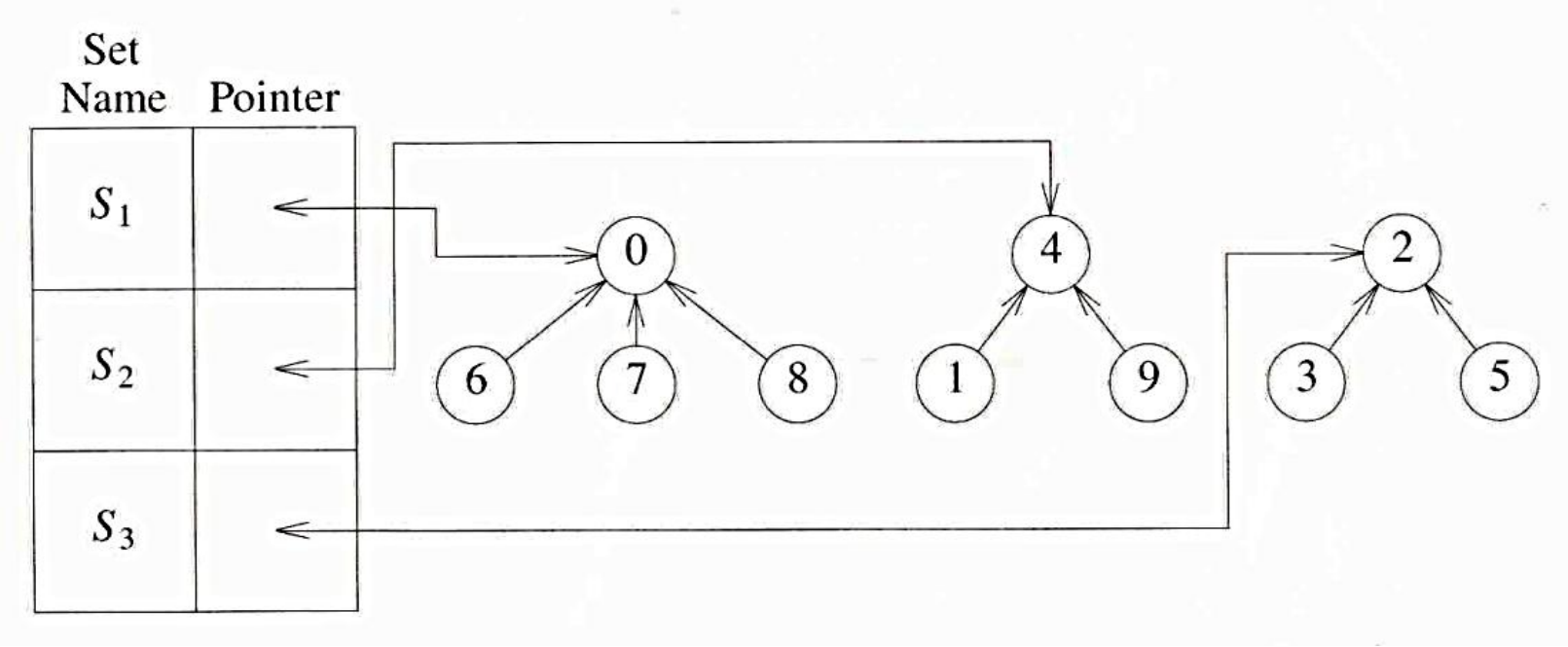




분리 집합의 표현 방법

- **(예)** 분리 집합 S_1, S_2, S_3에 대한 데이터 표현 방법:

$$S_1=\{0, 6, 7, 8\},\quad S_2=\{1, 4, 9\},\quad S_3=\{2, 3, 5\}$$

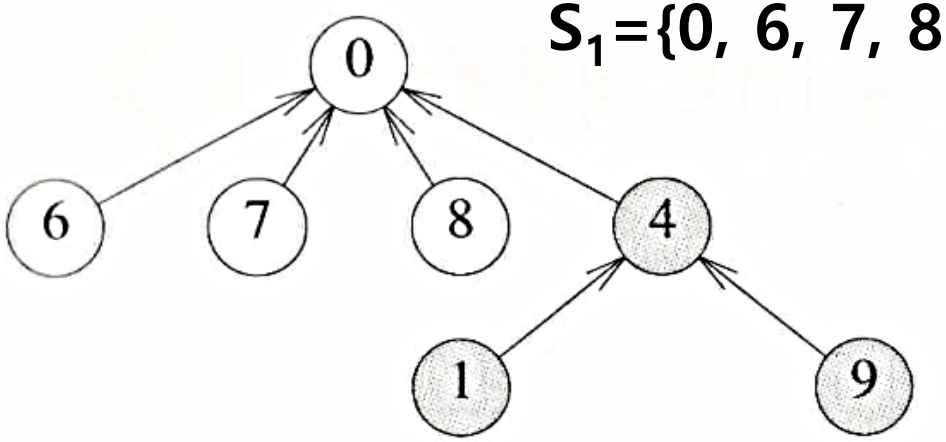


i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-1	4	-1	2	-1	2	0	0	0	4

Union 연산

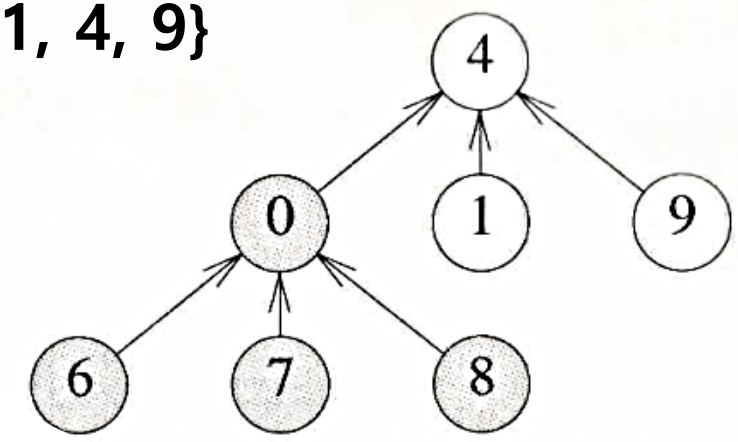
- **union(i,j)**
  - ➢두 집합 S_i와 S_j의 합집합을 만든다.
  - ➢이를 위해, 해당 집합을 나타내는 두 트리 중에서 하나를 선택하여 다른 트리의 서브 트리로 만든다.
  - ➢(예) 집합 S_1과 S_2에 대한 합집합 $S_1 \cup S_2$ 의 표현 방법:

S_1={0, 6, 7, 8}, S_2={1, 4, 9}



$S_1 \cup S_2$

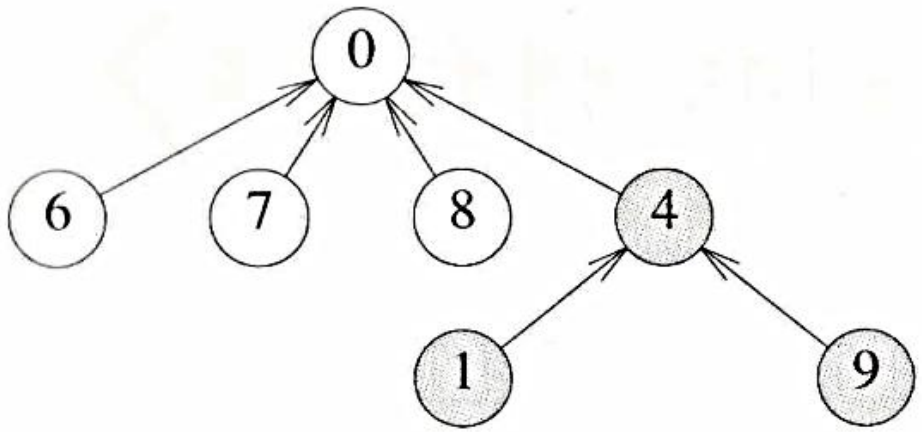
or



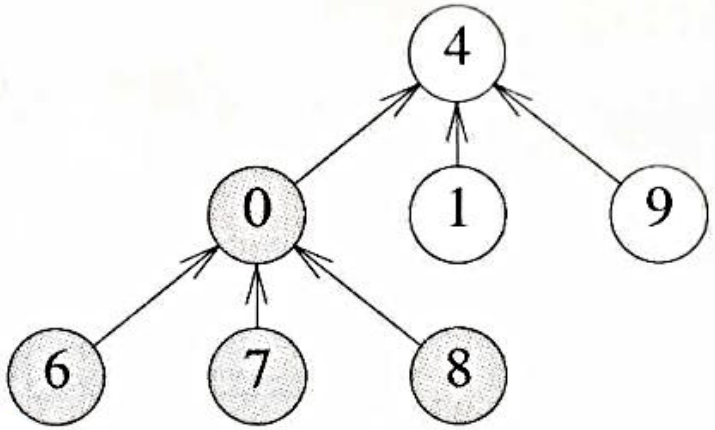
$S_1 \cup S_2$

Find 연산

- **find(i)**
  - 원소 i를 포함하는 집합을 탐색하고, 해당 집합에 해당하는 트리의 **루트 노드**를 반환한다.
  - (예) 집합 $S_1 \cup S_2$ 에서 원소 "8"에 대한 find(8)의 수행 결과:



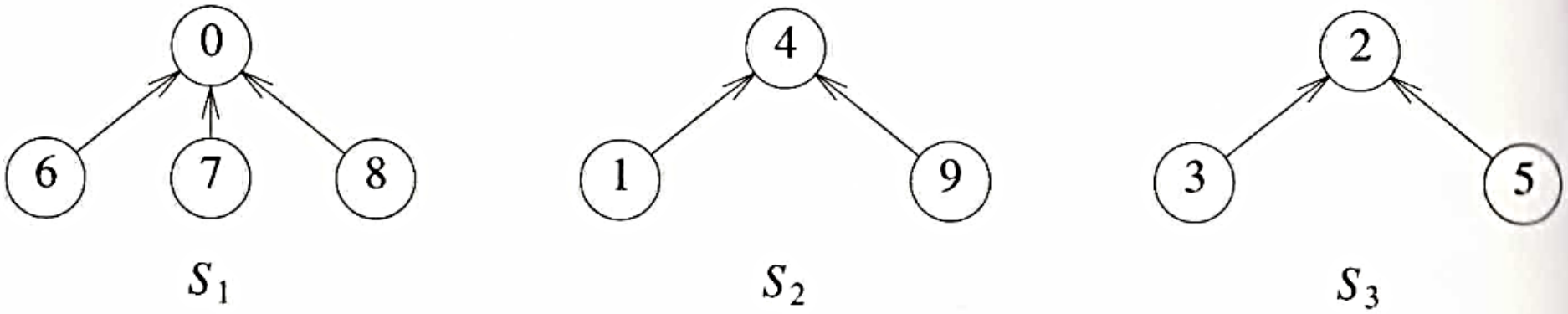
find(8) = 0



find(8) = 4

간단한 Union-Find 연산

```
void union(int i, int j){
      parent[i] = j;
}
int find(int i){
     for( ; parent[i] >= 0 ; i = parent[i] );
     return i;
}
```

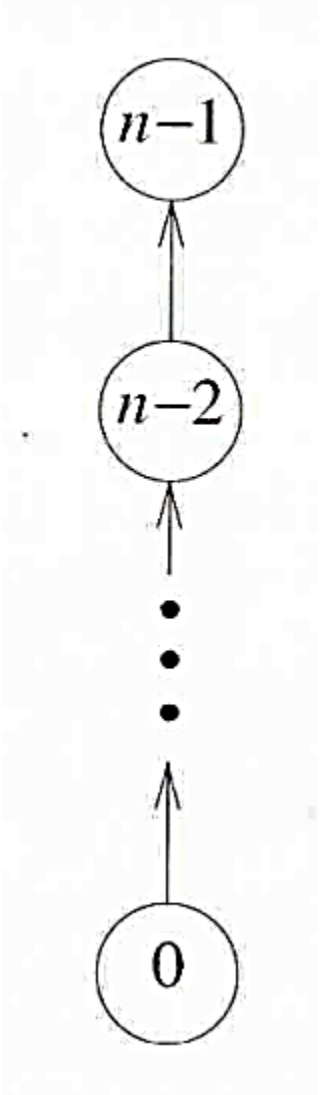


i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-1	4	-1	2	-1	2	0	0	0	4

간단한 Union-Find 연산의 문제점

- 아래의 Union-Find 연산을 연속적으로 수행하면 트리의 모양은 어떠한 형태가 되는가?

```
union(0, 1), find(0)
union(1, 2), find(0)
union(2, 3), find(0)
union(3, 4), find(0)
        ...
union(n-2, n-1), find(0)
```

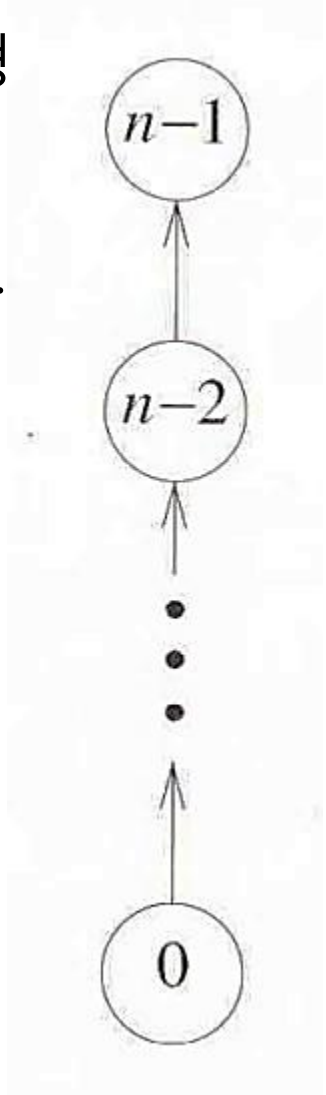


간단한 Union-Find 연산의 문제점

- 앞의 Union-Find 연산을 연속적으로 수행하면 오른쪽 그림과 같은 선형 구조 형태의 트리로 변형되는데 이를 "변질 트리"라 한다.
- 변질 트리에서 $n-1$번의 find를 수행하는 데 소요되는 시간은 $O(n^2)$이다.

```
union(0, 1), find(0)     →   1
union(1, 2), find(0)     →   2
union(2, 3), find(0)     →   3
            ...
union(n-2, n-1), find(0) → n-1
```

$= n(n-1)/2$



가중 규칙을 활용하여 개선한 Union 연산

- **union(i,j)**를 위한 가중 규칙
  - ➢ 루트 노드 i를 가진 트리의 노드 수가 루트 노드 j를 가진 트리의 노드 수보다 적으면 노드 j를 노드 i의 부모로 만들고, 그렇지 않으면 노드 i를 노드 j의 부모로 만든다.
  - ➢ 이를 위해 parent 배열에서 루트 노드에 해당하는 값은 "해당 트리가 포함하는 원소의 총 개수"를 음수로 표현하여 저장한다.

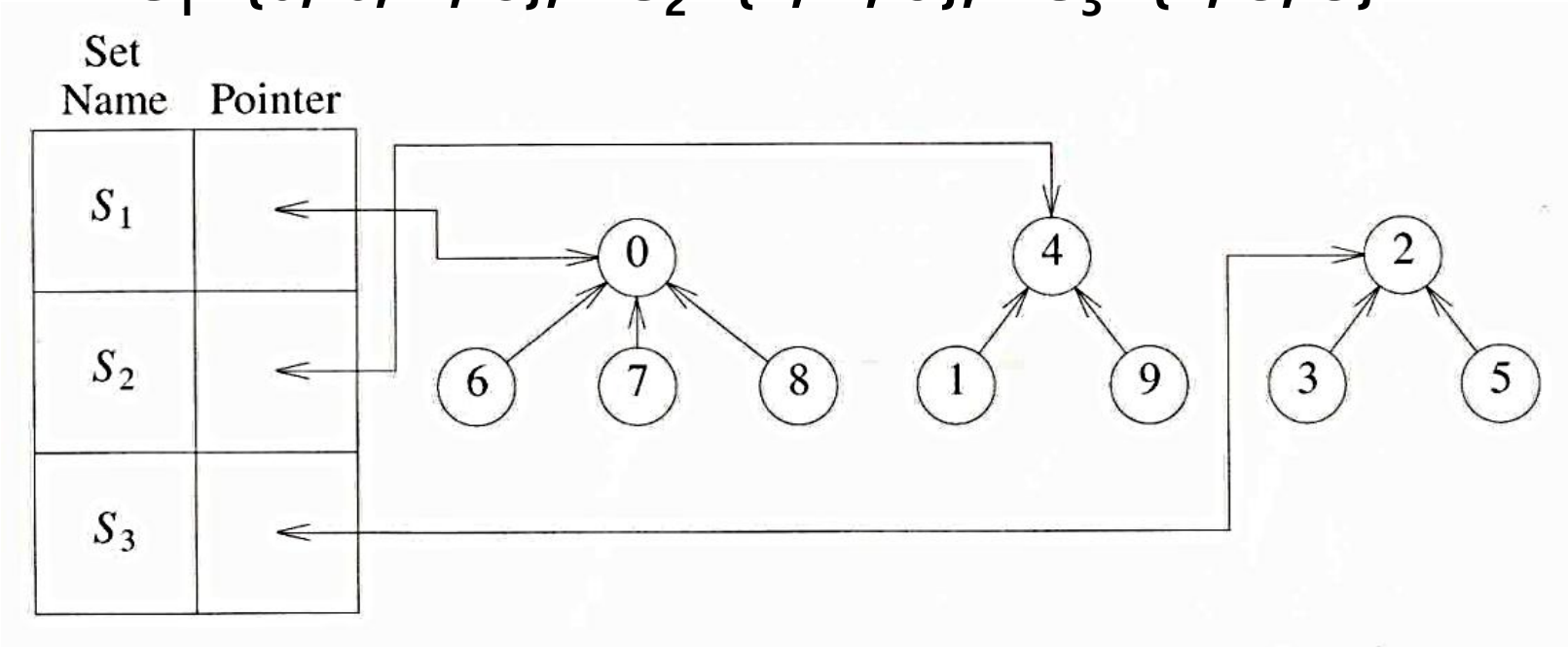
i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-1	4	-1	2	-1	2	0	0	0	4

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-4	4	-3	2	-3	2	0	0	0	4

가중 규칙을 활용하여 개선한 Union 연산

- (예) 분리 집합 S_1, S_2, S_3에 대한 개선된 데이터 표현 방법

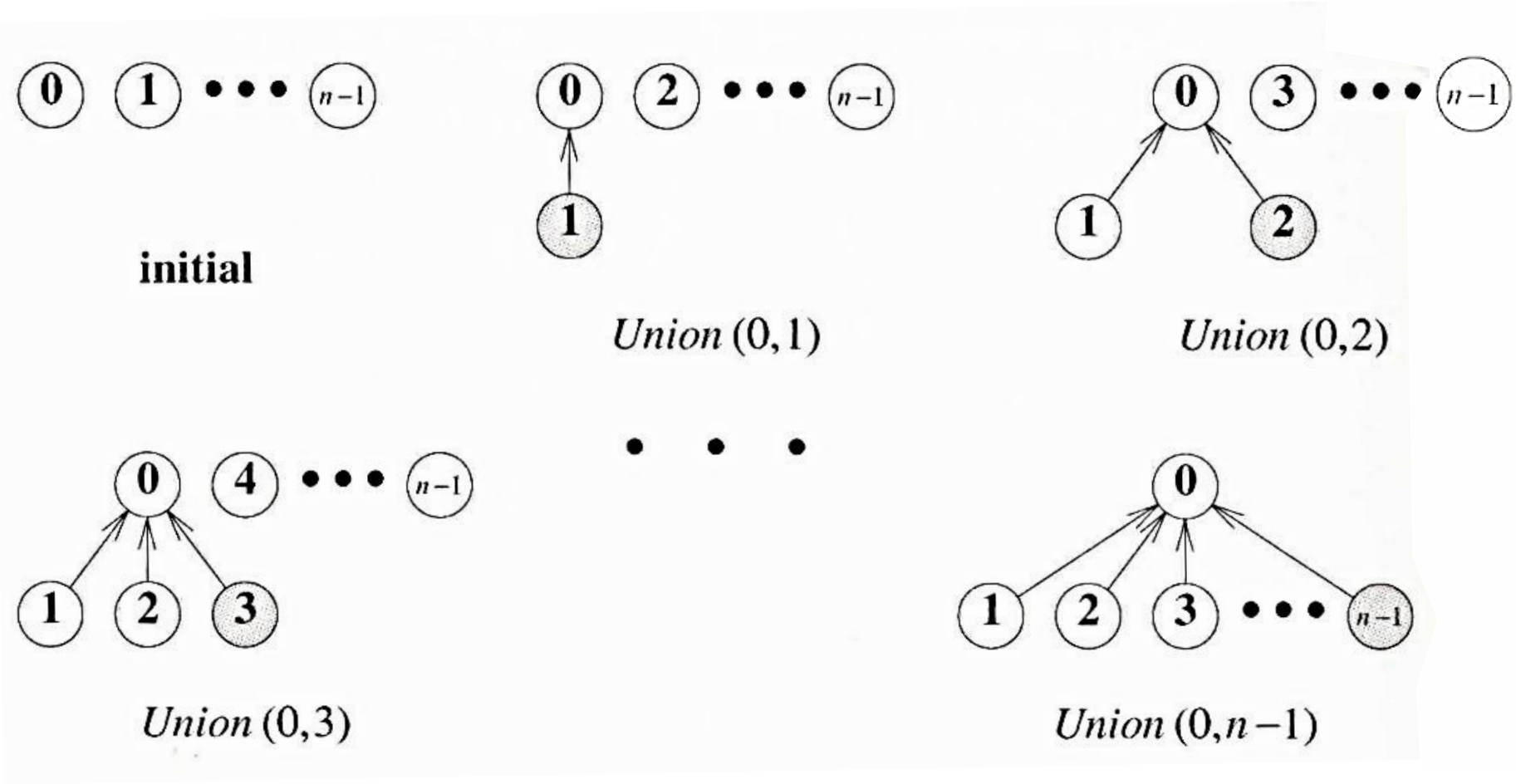
S_1={0, 6, 7, 8}, S_2={1, 4, 9}, S_3={2, 3, 5}



i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	**-4**	4	**-3**	2	**-3**	2	0	0	0	4

가중 규칙을 활용하여 개선한 Union 연산

- 가중 규칙을 적용하여 구한 트리 예제

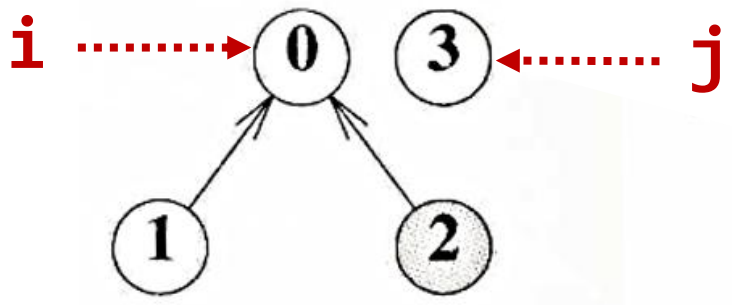


가중 규칙을 활용하여 개선한 Union 연산

```
void weighted_union(int i, int j){
      int temp = parent[i] + parent[j];
      if( -parent[i] < -parent[j] ){
            parent[i] = j;
            parent[j] = temp;
       }else{
            parent[j] = i;
            parent[i] = temp;
       }
}
```

노드 i와 j는 각 집합의 루트 노드를 의미함.

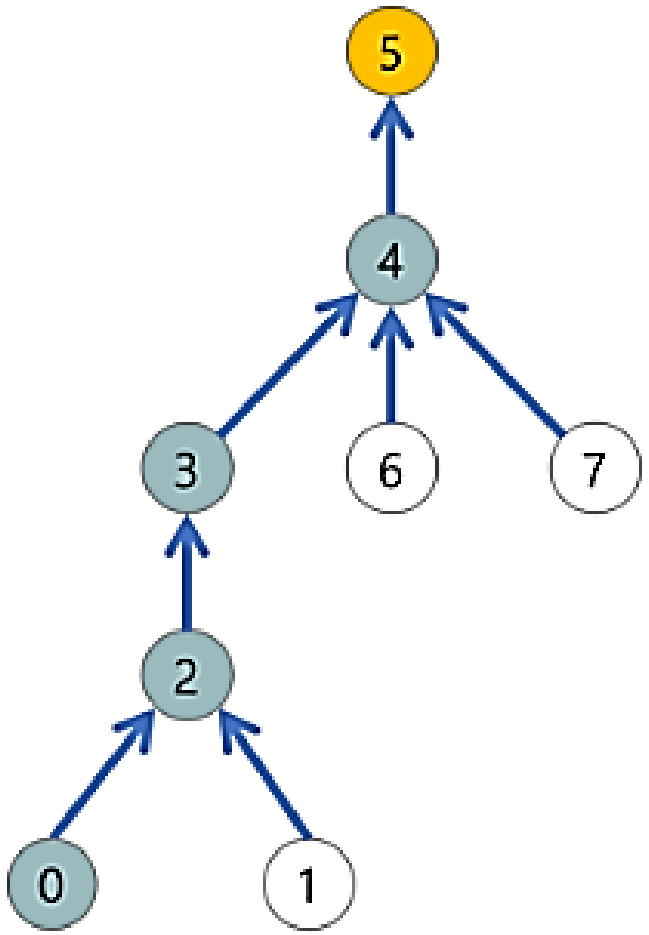
루트 노드 i와 j에 대한 **parent[i], parent[j]**는 각 집합이 포함하는 원소의 총 개수가 **음수**로 저장되어 있음.



Union (0,2)

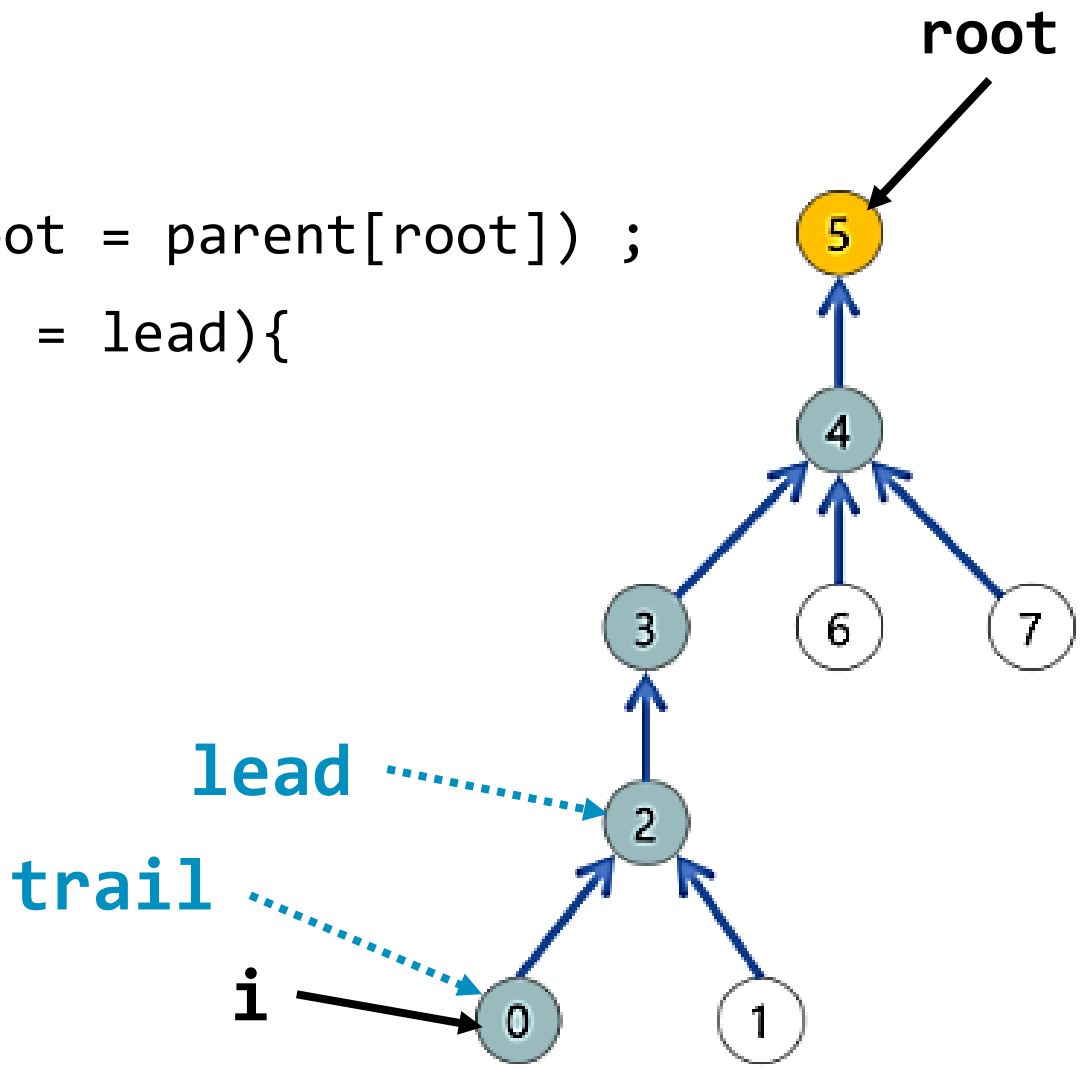
붕괴 규칙 (collapsing rule)

- find(i)를 위한 붕괴 규칙
  - 만일 노드 j가 노드 i에서 루트로 가는 경로상에 있으면서 parent[i] != root(i)이면 parent[j]를 root(i)로 지정한다.
- 붕괴 규칙의 역할
  - find(i) 연산 수행 시 find 연산의 목표가 되는 노드 i의 부모 노드를 루트 노드의 자식 노드로 지정한다.
  - 해당 작업을 노드 i부터 루트 노드까지의 경로상에 있는 모든 노드에 대하여 반복적으로 수행한다.
  - 이에 따라 트리의 높이를 낮춤으로써 변질 트리가 되는 현상을 방지할 수 있다.



붕괴 규칙을 활용하여 개선한 Find 연산

```
int collapsing_find(int i){
    int root, trail, lead;
    for(root = i; parent[root] >= 0; root = parent[root]) ;
    for(trail = i; trail != root; trail = lead){
          lead = parent[trail];
          parent[trail] = root;
     }
     return root;
}
```



Q & A